UBUNTU · LINUX · TERMINAL

Guía de estudio

Comandos de Linux

Cada comando con su función y un ejemplo de cómo funciona.
Pensada para practicar en Ubuntu, de cero a administración diaria.

Incluye navegación, archivos, permisos, usuarios, búsqueda, texto, procesos, disco, red, apt, systemd, logs, cron y operadores del shell. Más un plan de 7 días y una lista de errores que rompen sistemas.

Estudia en ~/practica · Consulta man y --help · 2026

Índice

01 Cómo usar esta guía (3)
02 Navegación (5)
03 Inspeccionar archivos y rutas (8)
04 Crear y borrar (8)
05 Copiar, mover y enlaces (6)
06 Ver el contenido de archivos (11)
07 Editores en terminal (3)
08 Permisos y propiedad (6)
09 Usuarios y grupos (11)
10 Buscar archivos y texto (7)
11 Procesar texto (16)
12 Comparar archivos e integridad (5)
13 Comprimir y empaquetar (4)
14 Procesos (12)
15 Memoria, CPU y disco (8)
16 Discos, particiones y montaje (6)
17 Red (10)
18 Acceso remoto y transferencia (5)
19 Paquetes en Ubuntu (apt y dpkg) (8)
20 systemd: servicios y arranque (6)
21 Registros (logs) (3)
22 Fecha, hora y espera (4)
23 Shell, variables e historial (8)
24 Redirecciones, tuberías y operadores (8)
25 Tareas programadas (3)
26 Apagar, reiniciar y salir (3)
27 Información del sistema y hardware (6)
28 Ayuda e información de comandos (5)
29 Otros coreutils que debes reconocer (13)
30 Plan de 7 días (7)
31 Errores comunes y reglas de oro (7)

El número entre paréntesis es la cantidad de fichas del capítulo. Usa los marcadores del PDF (panel izquierdo del lector) para saltar.

CAPÍTULO 01

Cómo usar esta guía

Esta guía reúne los comandos que se usan de verdad en Ubuntu y en Linux en general: GNU coreutils, administración del sistema, red, paquetes apt y systemd. No es cada binario de /usr/bin (hay cientos de herramientas opcionales), sino el conjunto completo con el que se estudia, trabaja y administra un sistema.

Cómo estudiar: abre una terminal, crea una carpeta de práctica y ejecuta cada ejemplo. Lee la salida. Cambia una opción. Vuelve a ejecutar. Eso graba el comando mejor que memorizar listas.

Carpeta de práctica

Crea un sitio seguro para experimentar. Nunca practiques rm, chmod o sudo en carpetas del sistema.

```
$ mkdir -p ~/practica && cd ~/practica
$ pwd
/home/usuario/practica
```

Todos los ejemplos de esta guía asumen que estás dentro de ~/practica, salvo que se indique otra ruta.

man / --help

Cada comando tiene su propio manual. Si olvidas una opción, no improvises: consulta.

```
$ man ls
$ ls --help | head
```

En man: flechas o espacio para bajar, /texto para buscar, q para salir.

Anatomía de un comando

La forma general es: comando [opciones] [argumentos]. El espacio separa cada pieza.

```
$ ls -la /etc
#  ls   = comando
#  -la  = opciones (largo + ocultos)
#  /etc = argumento (la carpeta)
```

Un guion corto (-l) es una opción corta. Dos guiones (--all) es la forma larga.

CAPÍTULO 02

Navegación

Si no sabes moverte, el resto de comandos no sirven. Empieza aquí hasta hacerlo sin pensar.

pwd

Print Working Directory. Muestra la ruta absoluta de la carpeta donde estás.

```
$ pwd
/home/usuario/practica
```

Si te pierdes, pwd es el primer comando que debes teclear.

cd

Change Directory. Entra a otra carpeta. Sin argumentos vuelve a tu home.

```
$ cd /etc
$ pwd
/etc
$ cd ~
$ pwd
/home/usuario
$ cd -
/etc
```

Atajos: ~ home, . carpeta actual, .. padre, - carpeta anterior, / raíz del sistema.

ls

Lista el contenido de una carpeta. Es el comando que más vas a usar.

```
$ ls
backup  frutas.txt  notas.txt
$ ls -la
drwxr-xr-x  3 usuario usuario 4096 sep 22 10:00 .
-rw-r--r--  1 usuario usuario   12 sep 22 10:01 notas.txt
```

-l detalle, -a ocultos, -h tamaños legibles, -t por fecha, -R recursivo. Combínalos: ls -lah.

tree

Dibuja el árbol de carpetas. En Ubuntu se instala con apt.

```
$ sudo apt install tree
$ tree ~/practica
/home/usuario/practica
├── backup
│   └── notas-copia.txt
└── frutas.txt
```

tree -L 2 limita la profundidad. tree -d muestra solo carpetas.

pushd / popd / dirs

Apilan carpetas para ir y volver sin reescribir rutas largas.

```
$ pwd
/home/usuario/practica
$ pushd /etc
/etc ~/practica
$ popd
~/practica
```

dirs -v lista la pila numerada. Útil cuando saltas entre 2 o 3 proyectos.

CAPÍTULO 03

Inspeccionar archivos y rutas

Antes de editar o borrar, identifica qué es cada cosa: archivo, carpeta, enlace o binario.

`file`

Adivina el tipo real del archivo (no solo la extensión).

```
$ file notas.txt
notas.txt: ASCII text
$ file /bin/ls
/bin/ls: ELF 64-bit LSB pie executable
```

Una foto renombrada a .txt sigue siendo imagen. file lo detecta.

`stat`

Muestra metadatos: tamaño, dueño, permisos, fechas de acceso y modificación.

```
$ stat notas.txt
  File: notas.txt
  Size: 12         Blocks: 8     IO Block: 4096  regular file
Access: (0644/-rw-r--r--)  Uid: (1000/usuario)
```

Más completo que ls -l. Sirve para depurar permisos y fechas.

`basename`

Devuelve solo el nombre final de una ruta.

```
$ basename /home/usuario/practica/notas.txt
notas.txt
```

Muy usado dentro de scripts para extraer el nombre de un archivo.

`dirname`

Devuelve la carpeta de una ruta, sin el archivo.

```
$ dirname /home/usuario/practica/notas.txt
/home/usuario/practica
```

Complemento de basename.

realpath

Resuelve . .. y enlaces simbólicos y entrega la ruta absoluta real.

```
$ realpath ./notas.txt
/home/usuario/practica/notas.txt
```

Útil cuando un script recibe rutas relativas y necesita la canónica.

readlink

Muestra a dónde apunta un enlace simbólico.

```
$ ln -s notas.txt atajo.txt
$ readlink atajo.txt
notas.txt
$ readlink -f atajo.txt
/home/usuario/practica/notas.txt
```

-f resuelve la cadena completa hasta el archivo final.

namei

Descompone una ruta y muestra de qué tipo es cada eslabón.

```
$ namei -l /home/usuario/practica/notas.txt
f: /home/usuario/practica/notas.txt
dr-xr-xr-x root    root    /
drwxr-xr-x usuario usuario practica
```

Sirve para cazar un permiso roto a mitad de una ruta larga.

lsattr / chattr

Atributos extra de ext4, independientes de rwx. +i hace un archivo inmutable.

```
$ sudo chattr +i importante.txt
$ lsattr importante.txt
----i--------- importante.txt
$ sudo chattr -i importante.txt
```

Un archivo con +i no se borra ni se edita, ni siquiera con root, hasta quitar el atributo.

CAPÍTULO 04

Crear y borrar

Crea estructura con mkdir y touch. Borra solo cuando sepas exactamente qué estás borrando.

mkdir

Crea una o más carpetas. -p crea padres que falten y no falla si ya existen.

```
$ mkdir curso
$ mkdir -p curso/linux/dia1
$ ls curso/linux
dia1
```

Sin -p, mkdir curso/linux falla si curso no existe.

rmdir

Borra una carpeta solo si está vacía. Es más seguro que rm -r.

```
$ mkdir vacia
$ rmdir vacia
```

Si tiene archivos, usa rm -r después de revisar el contenido.

touch

Crea un archivo vacío o actualiza su fecha de modificación si ya existe.

```
$ touch diario.txt
$ ls -l diario.txt
-rw-r--r-- 1 usuario usuario 0 sep 22 10:05 diario.txt
```

También se usa para refrescar la fecha: touch archivo_ya_existente.

rm

Borra archivos. -r borra carpetas. -i pregunta. -f no pregunta.

```
$ rm diario.txt
$ rm -r curso
```

*NUNCA ejecutes rm -rf / ni rm -rf * fuera de ~/practica. No hay papelera.*

shred

Sobrescribe un archivo antes de borrarlo, para que cueste recuperarlo.

```
$ echo secreto > clave.txt
$ shred -u -z -n 3 clave.txt
```

-u borra al terminar, -z pone ceros al final, -n 3 hace 3 pasadas. No es magia en SSD.

unlink

Elimina un nombre de archivo (un enlace). Equivale a rm de un solo archivo.

```
$ touch tmp.dat
$ unlink tmp.dat
```

Solo admite un archivo. Para varios, usa rm.

mktemp

Crea un archivo o carpeta temporal con nombre aleatorio, seguro para scripts.

```
$ tmp=$(mktemp)
$ echo $tmp
/tmp/tmp.xK91ab
$ rm $tmp
```

mktemp -d crea una carpeta temporal.

truncate

Cambia el tamaño de un archivo. Puede vaciarlo o agrandarlo.

```
$ echo hola > demo.txt
$ truncate -s 0 demo.txt
$ wc -c demo.txt
0 demo.txt
```

truncate -s 1M disco.img crea un archivo de 1 mebibyte.

CAPÍTULO 05

Copiar, mover y enlaces

cp duplica, mv cambia de sitio o de nombre, ln crea otro nombre que apunta al mismo contenido.

cp

Copia archivos. -r copia carpetas. -v muestra lo que hace. -i pregunta antes de pisar.

```
$ cp notas.txt backup-notas.txt
$ cp -r curso/ curso-copia/
```

cp origen destino. Si destino es carpeta, copia dentro. cp -a preserva permisos y fechas.

mv

Mueve o renombra. En el mismo disco es instantáneo: solo cambia la ruta.

```
$ mv backup-notas.txt notas-antigua.txt
$ mv notas-antigua.txt backup/
```

No hay comando rename básico en todos los sistemas. mv archivo.txt nuevo.txt renombra.

ln

Crea un enlace duro: dos nombres para los mismos datos en disco.

```
$ echo dato > original.txt
$ ln original.txt duro.txt
$ echo cambio >> original.txt
$ cat duro.txt
dato
cambio
```

Si borras original.txt, duro.txt sigue teniendo el contenido. No funciona entre discos distintos.

ln -s

Crea un enlace simbólico (atajo). Apunta a una ruta, no a los datos.

```
$ ln -s /var/log logs
$ ls -l logs
lrwxrwxrwx 1 usuario usuario 8 sep 22 10:10 logs -> /var/log
```

Si mueves el destino, el enlace se rompe. Es el tipo de enlace que más se usa.

`install`

Copia un archivo y de paso le pone dueño y permisos. Típico al instalar un script.

```
$ install -m 755 script.sh /usr/local/bin/miscript
```

Más control que cp cuando dejas un programa en el PATH.

`rsync`

Copia incremental, local o remota. Solo transfiere lo que cambió.

```
$ rsync -av curso/ curso-backup/
sending incremental file list
dia1/
dia1/apuntes.txt
```

-a archivo (permisos, fechas), -v verbose, -n simula, --delete iguala borrando el exceso.

CAPÍTULO 06

Ver el contenido de archivos

Para archivos pequeños usa cat. Para logs largos usa less. Para las últimas líneas, tail.

cat

Concatenate. Muestra uno o varios archivos enteros en pantalla.

```
$ echo hola > a.txt
$ echo mundo > b.txt
$ cat a.txt b.txt
hola
mundo
```

También une archivos: cat a.txt b.txt > juntos.txt. No lo uses con binarios.

tac

Igual que cat, pero de abajo hacia arriba (cat al revés).

```
$ printf 'uno\ndos\ntres\n' > nums.txt
$ tac nums.txt
tres
dos
uno
```

Útil para ver el final de un log corto sin tail.

less

Visor por páginas. No carga el archivo entero en memoria.

```
$ less /var/log/syslog
```

Espacio avanza, b retrocede, /busca, n siguiente, q sale. Prefiérelo a more.

more

Visor antiguo, solo avanza. En la práctica usa less.

```
$ more /etc/hosts
```

Lo encontrarás en scripts viejos y en documentación clásica.

head

Muestra las primeras líneas. Por defecto 10.

```
$ seq 20 > veinte.txt
$ head -n 3 veinte.txt
1
2
3
```

head -c 20 archivo muestra los primeros 20 bytes.

tail

Muestra las últimas líneas. -f sigue el archivo en vivo (logs).

```
$ tail -n 2 veinte.txt
19
20
$ tail -f /var/log/syslog
```

Ctrl+C sale de tail -f. Imprescindible para depurar servicios.

nl

Numera las líneas de un archivo.

```
$ nl frutas.txt
     1  manzana
     2  pera
     3  platano
```

Útil para citar una línea concreta al hablar de un error.

wc

Word count. Cuenta líneas, palabras y bytes.

```
$ echo 'hola linux ubuntu' > frase.txt
$ wc frase.txt
 1  3 18 frase.txt
```

wc -l solo líneas, -w palabras, -c bytes, -m caracteres.

od / hexdump

Vuelca un archivo en octal o hexadecimal. Sirve para ver bytes ocultos.

```
$ printf 'ABC\n' | od -c
0000000   A   B   C  \n
```

hexdump -C es más legible para inspección binaria.

strings

Extrae cadenas de texto legibles de un binario.

```
$ strings /bin/ls | head
...
```

Útil para inspeccionar un programa sin abrirlo en un editor hexadecimal.

tee

Copia la entrada a un archivo y también la deja pasar por la tubería.

```
$ echo 'backup ok' | tee registro.txt
backup ok
$ cat registro.txt
backup ok
```

tee -a añade al final. Ver y guardar al mismo tiempo.

CAPÍTULO 07

Editores en terminal

Tienes que poder cambiar un archivo sin entorno gráfico. Empieza por nano; luego aprende a salir de vim.

nano

Editor sencillo. Los atajos aparecen abajo. Ideal para empezar.

```
$ nano notas.txt
# Escribes texto
# Ctrl+O guarda, Enter confirma el nombre
# Ctrl+X sale
```

Ctrl+K corta la línea, Ctrl+U pega, Ctrl+W busca.

vim

Editor potente, modal. Por defecto estás en modo normal, no puedes escribir hasta pulsar i.

```
$ vim notas.txt
# i  = insertar texto
# Esc = volver a modo normal
# :w  = guardar
# :q  = salir
# :wq = guardar y salir
# :q! = salir sin guardar
```

Si vim 'te atrapa', pulsa Esc y escribe :q! Enter. Practica solo esos seis atajos al principio.

echo y redirección

Para cambios de una línea no siempre hace falta un editor.

```
$ echo 'primera linea' > notas.txt
$ echo 'segunda linea' >> notas.txt
$ cat notas.txt
primera linea
segunda linea
```

> crea o pisa el archivo. >> añade al final. Confundirlos borra trabajo.

CAPÍTULO 08

Permisos y propiedad

Linux decide quién puede leer, escribir o ejecutar. Si un script 'no funciona', casi siempre son permisos.

ls -l (leer permisos)

La primera columna es el mapa de permisos: tipo + dueño + grupo + otros.

```
$ ls -l script.sh
-rwxr-xr-x 1 usuario usuario 120 sep 22 11:00 script.sh
```

- archivo, d carpeta, l enlace. r=4 w=2 x=1. rwxr-xr-x = 755. rw-r--r-- = 644.

chmod

Cambia permisos. Puedes usar números (755) o letras (u+x).

```
$ touch script.sh
$ chmod 755 script.sh
$ ls -l script.sh
-rwxr-xr-x 1 usuario usuario 0 sep 22 11:01 script.sh
$ chmod u+x,g-w archivo.txt
```

u usuario, g grupo, o others, a all. + añade, - quita, = asigna. Evita 777.

chown

Cambia el dueño y, si quieres, el grupo. Casi siempre requiere sudo.

```
$ sudo chown usuario:usuario archivo.txt
$ sudo chown -R usuario:usuario ~/practica
```

La forma dueño:grupo es la más habitual. -R aplica a toda la carpeta.

chgrp

Cambia solo el grupo del archivo.

```
$ sudo chgrp sudo notas.txt
$ ls -l notas.txt
-rw-r--r-- 1 usuario sudo 12 sep 22 11:02 notas.txt
```

Más preciso que chown cuando el dueño ya es correcto.

umask

Máscara de permisos por defecto para archivos y carpetas nuevos.

```
$ umask
0022
$ umask -S
u=rwx,g=rx,o=rx
```

0022 → archivos 644 y carpetas 755. 0077 → solo tú puedes leer lo nuevo.

getfacl / setfacl

ACL: permisos extra para usuarios concretos, más finos que rwx clásico.

```
$ sudo setfacl -m u:ana:r notas.txt
$ getfacl notas.txt
user::rw-
user:ana:r--
group::r--
```

Aparece una + al final de ls -l cuando hay ACL.

CAPÍTULO 09

Usuarios y grupos

Linux es multi-usuario. Tu cuenta es un UID; sudo te presta poderes de root un comando cada vez.

whoami

Imprime el usuario con el que se ejecuta la sesión actual.

```
$ whoami
usuario
```

Después de sudo -i cambiará a root. Compruébalo siempre.

id

Muestra UID, GID y todos los grupos del usuario.

```
$ id
uid=1000(usuario) gid=1000(usuario) grupos=1000(usuario),27(sudo),100(users)
```

Si un permiso falla, id te dice a qué grupos perteneces de verdad.

who / w / users

Quién está conectado ahora. w añade qué está haciendo y desde cuándo.

```
$ who
usuario  tty2  2026-09-22 09:01
$ w
```

Útil en servidores compartidos.

last / lastlog

Historial de inicios de sesión. lastlog es la última vez de cada cuenta.

```
$ last -n 5
usuario  tty2  still logged in
$ lastlog | head
```

Sirve para auditar accesos extraños.

sudo

Ejecuta un comando como administrador. Te pedirá tu contraseña, no la de root.

```
$ sudo apt update
[sudo] password for usuario:
```

sudo -i abre una shell de root. sudo !! repite el comando anterior con sudo.

su

Cambia de usuario. su - usuario carga su entorno completo.

```
$ su - otro
Password:
```

En Ubuntu moderno se prefiere sudo. root suele tener el login deshabilitado.

passwd

Cambia la contraseña de tu usuario. Con sudo, la de otro.

```
$ passwd
Current password:
New password:
```

sudo passwd usuario resetea la de esa cuenta.

adduser / useradd

Crea un usuario. En Ubuntu usa adduser: te hace preguntas y crea el home.

```
$ sudo adduser maria
$ sudo useradd -m -s /bin/bash pedro
```

useradd es más bajo nivel y no crea home salvo que pongas -m.

usermod / userdel

Modifica o elimina una cuenta. -aG añade a un grupo sin quitar los demás.

```
$ sudo usermod -aG sudo maria
$ sudo userdel -r pedro
```

Sin -a, -G reemplaza todos los grupos extra. Es un error clásico.

groupadd / groupdel / groups

Administra grupos. groups lista los grupos de un usuario.

```
$ sudo groupadd proyecto
$ groups usuario
usuario sudo
```

Los grupos controlan acceso compartido a carpetas de equipo.

visudo

Edita /etc/sudoers de forma segura: valida la sintaxis antes de guardar.

```
$ sudo visudo
```

Nunca edites sudoers con nano a pelo: un error te deja sin sudo.

CAPÍTULO 10

Buscar archivos y texto

Hay dos búsquedas distintas: find localiza archivos; grep localiza texto dentro de archivos.

find

Recorre el árbol de carpetas y filtra por nombre, tipo, tamaño o fecha.

```
$ find ~/practica -name '*.txt'
/home/usuario/practica/notas.txt
$ find . -type d
$ find . -size +1M
```

-name distingue mayúsculas; -iname no. -type f archivo, d carpeta. -mtime -1 últimos 1 día.

locate / updatedb

Busca por una base de datos, mucho más rápido que find. Puede estar desactualizado.

```
$ sudo updatedb
$ locate notas.txt
/home/usuario/practica/notas.txt
```

En Ubuntu el paquete es plocate o mlocate. No ve archivos creados hace 1 minuto hasta updatedb.

which

Muestra qué ejecutable se usaría si tecleas ese comando.

```
$ which python3
/usr/bin/python3
```

Si hay varios python, which te dice cuál gana según el PATH.

whereis

Busca binario, código fuente y página man de un comando.

```
$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

Más amplio que which.

type

Dice si un nombre es comando, alias, función o builtin del shell.

```
$ type ls
ls is aliased to `ls --color=auto'
$ type cd
cd is a shell builtin
```

Resuelve la duda '¿por qué este ls se comporta distinto?'.

grep

Busca un patrón de texto. Es el comando estrella junto con las tuberías.

```
$ echo -e 'error disco\nok\nERROR red' > log.txt
$ grep -i error log.txt
error disco
ERROR red
```

-i ignora mayúsculas, -n número de línea, -R recursivo, -v invierte, -c cuenta.

grep con tubería

El uso más habitual: filtrar la salida de otro comando.

```
$ ps aux | grep ssh
$ dmesg | grep -i error
```

grep 'palabra' archivo busca en archivo. comando | grep busca en la salida.

CAPÍTULO 11

Procesar texto

Linux trata casi todo como texto. Encadena estos filtros con | y resuelves el 80% de las tareas.

echo / printf

Imprime texto. printf da formato preciso (saltos, columnas, números).

```
$ echo hola
hola
$ printf '%s %d\n' ubuntu 24
ubuntu 24
```

echo -e interpreta \n. En scripts serios se prefiere printf.

sort

Ordena líneas. -n orden numérico, -r inverso, -k columna, -u únicos.

```
$ printf '10\n2\n1\n' | sort -n
1
2
10
```

Sin -n, 10 iría antes que 2 porque ordena como texto.

uniq

Quita líneas repetidas consecutivas. Casi siempre va detrás de sort.

```
$ printf 'a\na\nb\na\n' | sort | uniq
a
b
```

uniq -c cuenta repeticiones. uniq solo no detecta duplicados si no están juntos.

cut

Corta columnas de cada línea, por carácter o por delimitador.

```
$ echo 'ana:x:1001:1001' | cut -d: -f1,3
ana:1001
```

-d delimitador, -f campos. Ideal para /etc/passwd y CSV simples.

paste

Pega archivos en columnas, línea a línea.

```
$ echo -e 'a\nb' > x.txt
$ echo -e '1\n2' > y.txt
$ paste x.txt y.txt
a       1
b       2
```

El inverso informal de cut.

tr

Traduce o borra caracteres. No entiende regex: trabaja letra a letra.

```
$ echo 'Hola Linux' | tr 'A-Z' 'a-z'
hola linux
$ echo 'a  b   c' | tr -s ' '
a b c
```

-d borra. tr -d '\n' quita saltos de línea. -s comprime repeticiones.

sed

Editor de flujo. Cambia texto al vuelo, línea por línea, sin abrir el archivo.

```
$ echo 'me gusta windows' | sed 's/windows/linux/'
me gusta linux
$ sed -i 's/hola/adios/g' notas.txt
```

s/viejo/nuevo/ sustituye. g = todas en la línea. -i edita el archivo (haz copia antes).

awk

Procesa columnas. El comodín cuando cut se queda corto.

```
$ echo 'ana 8 9' | awk '{print $1, ($2+$3)/2}'
ana 8.5
$ awk -F: '{print $1}' /etc/passwd | head
```

$1 primera columna, NF número de campos, NR número de línea. -F cambia el separador.

xargs

Convierte líneas de entrada en argumentos de otro comando.

```
$ find . -name '*.tmp' | xargs rm -f
$ echo 'a.txt b.txt' | xargs touch
```

xargs -n 1 ejecuta una vez por ítem. xargs -I {} rm {} usa un marcador.

split / csplit

Parte un archivo grande en trozos. split por tamaño o líneas; csplit por patrón.

```
$ seq 100 > cien.txt
$ split -l 40 cien.txt parte_
$ ls parte_*
parte_aa  parte_ab  parte_ac
```

cat parte_* > cien.txt vuelve a unirlos.

column

Alinea texto en columnas bonitas.

```
$ printf 'nombre edad\nana 21\nlucas 19\n' | column -t
nombre  edad
ana     21
lucas   19
```

column -t -s: formatea archivos separados por dos puntos.

rev

Invierte cada línea carácter a carácter.

```
$ echo linux | rev
xunil
```

Truco de scripts y ejercicios de terminal, poco uso diario.

seq

Genera secuencias de números. Perfecto para pruebas.

```
$ seq 3
1
2
3
$ seq 2 2 8
2
4
6
8
```

seq INICIO PASO FIN. Combínalo con xargs o bucles for.

shuf

Baraja líneas al azar.

```
$ seq 5 | shuf
3
1
5
2
4
```

shuf -n 1 elige un elemento aleatorio.

---

yes

Imprime una cadena sin parar. Se usa para responder automáticamente sí.

```
$ yes | head -n 3
y
y
y
```

yes | comando alimenta confirmaciones. Ctrl+C para detenerlo si se va de madre.

---

expr / test / [ ]

Aritmética y condiciones en shell. [[ ]] en bash es la forma moderna.

```
$ expr 3 + 4
7
$ test -f notas.txt && echo existe
existe
$ [ -d backup ] && echo carpeta
```

-f archivo, -d carpeta, -z cadena vacía, -eq igual numérico. En bash: [[-f notas.txt]].

CAPÍTULO 12

Comparar archivos e integridad

Para ver diferencias usa diff. Para comprobar que un archivo no se corrompió, usa sumas hash.

diff

Compara dos archivos línea a línea y muestra lo que cambió.

```
$ echo hola > a.txt
$ echo hala > b.txt
$ diff a.txt b.txt
1c1
< hola
---
> hala
```

diff -u formato unificado (el de git). diff -r compara carpetas.

cmp

Compara byte a byte. Solo te dice si son distintos y en qué posición.

```
$ cmp a.txt b.txt
a.txt b.txt differ: byte 2, line 1
```

Más bajo nivel que diff. Útil con binarios.

comm

Compara dos archivos YA ORDENADOS y reparte líneas en 3 columnas.

```
$ comm <(echo -e 'a\nb') <(echo -e 'b\nc')
a
                b
        c
```

Col1 solo en el primero, col2 solo en el segundo, col3 en ambos.

md5sum / sha256sum

Calcula una huella del archivo. Si cambia un bit, cambia el hash.

```
$ echo hola > a.txt
$ sha256sum a.txt
5891b5b5...  a.txt
```

Compara el hash publicado por un sitio con el de tu descarga. sha256 es el habitual.

cksum / sum / b2sum

Otras sumas de comprobación. cksum es clásico de UNIX; b2sum usa BLAKE2.

```
$ cksum a.txt
3015617425 5 a.txt
```

Para seguridad moderna, quédate con sha256sum o sha512sum.

CAPÍTULO 13

Comprimir y empaquetar

tar empaqueta. gzip, bzip2 y xz comprimen. zip es el formato más compatible con Windows.

tar

Tape archive. Junta muchos archivos en uno. Suele ir con gzip: .tar.gz

```
$ tar -czvf curso.tar.gz curso/
$ tar -tzvf curso.tar.gz
$ tar -xzvf curso.tar.gz
```

-c crear, -x extraer, -t listar, -z gzip, -j bzip2, -J xz, -v verbose, -f archivo.

gzip / gunzip / zcat

Comprime un archivo suelto a .gz y deja el original reemplazado.

```
$ echo texto | gzip > nota.gz
$ zcat nota.gz
texto
$ gunzip nota.gz
```

gzip archivo crea archivo.gz y borra archivo. gzip -k lo conserva.

bzip2 / xz

Más compresión que gzip (más lentos). Extensiones .bz2 y .xz.

```
$ xz -k grande.log
$ unxz grande.log.xz
```

xz -k conserva el original. En discos llenos, xz suele ganar espacio.

zip / unzip

Formato zip, el que entiende cualquier sistema.

```
$ zip -r curso.zip curso/
$ unzip -l curso.zip
$ unzip curso.zip -d destino/
```

zip -r incluye subcarpetas. unzip -l lista sin extraer.

CAPÍTULO 14

Procesos

Un proceso es un programa en ejecución. Tiene PID, dueño, CPU y memoria. Tú puedes listarlo o matarlo.

ps

Fotografía de los procesos. aux es el juego de opciones más usado.

```
$ ps aux | head
USER  PID %CPU %MEM COMMAND
root    1  0.0  0.2 /sbin/init
```

ps -ef estilo System V. ps aux --sort=-%cpu los que más CPU comen.

top / htop

Monitor en vivo. top viene instalado; htop es más cómodo (apt install htop).

```
$ top
$ htop
```

En top: P ordena por CPU, M por memoria, q sale, k mata un PID. htop se maneja con flechas.

pstree

Árbol de procesos: quién lanzó a quién.

```
$ pstree -p | head
systemd(1)-+-sshd(812)-+-sshd(1402)
           |-cron(790)
```

Ayuda a entender servicios y shells hijas.

pgrep / pidof

Buscan el PID de un programa por nombre.

```
$ pgrep -a sshd
812 /usr/sbin/sshd -D
$ pidof systemd
1
```

pgrep -a muestra también la línea de comando.

kill / killall / pkill

Envían una señal a un proceso. Por defecto TERM (cierre limpio). -9 es KILL (forzado).

```
$ sleep 300 &
[1] 4521
$ kill 4521
$ killall sleep
$ pkill -f 'python app.py'
```

Primero kill PID. Si no muere, kill -9 PID. -9 no deja guardar estado: úsalo de último recurso.

nice / renice

Prioridad de CPU. nice al lanzar; renice a un proceso ya en marcha. Rango -20 a 19.

```
$ nice -n 15 gzip enorme.log
$ renice 10 -p 4521
```

Número alto = más educado = menos prioridad. Solo root baja de 0.

nohup

Lanza un comando que sobrevive al cierre de la terminal.

```
$ nohup ./servidor.sh &
nohup: ignoring input and appending output to 'nohup.out'
```

La salida va a nohup.out. Alternativa moderna: systemd o tmux/screen.

jobs / fg / bg / disown

Control de trabajos de la shell actual. Un proceso en primer o segundo plano.

```
$ sleep 200 &
[1] 4600
$ jobs
[1]+  Running  sleep 200 &
$ fg %1
```

Ctrl+Z pausa. bg continúa en segundo plano. disown lo suelta de la shell.

timeout

Ejecuta un comando y lo mata si tarda más de N segundos.

```
$ timeout 2 sleep 10
$ echo $?
124
```

El código 124 significa que se agotó el tiempo. Ideal en scripts y cron.

watch

Repite un comando cada N segundos y refresca la pantalla.

```
$ watch -n 1 df -h
```

Ctrl+C sale. watch -d resalta lo que cambia.

time

Mide cuánto tarda un comando (real, usuario, sistema).

```
$ time sleep 1
real    0m1.002s
user    0m0.000s
sys     0m0.000s
```

En bash, time es un builtin. /usr/bin/time da más detalle.

lsof / fuser

Qué proceso tiene abierto un archivo o un puerto.

```
$ sudo lsof -i :22
sshd  812 root  3u  IPv4  TCP *:ssh (LISTEN)
$ sudo fuser -v /var/log/syslog
```

Clásico: 'device busy' al desmontar → fuser o lsof te dicen quién lo usa.

CAPÍTULO 15

Memoria, CPU y disco

Cuando el sistema va lento, mira CPU (top), RAM (free) y espacio (df). En ese orden.

free

Memoria RAM y swap. -h en unidades humanas.

```
$ free -h
       total  used  free  buff/cache  available
Mem:    7,6G  2,1G  1,4G        4,1G       5,2G
Swap:   2,0G  0,0B  2,0G
```

available es lo que importa, no free. Linux usa RAM libre como caché y eso es bueno.

uptime

Hace cuánto está encendida la máquina y la carga media (1, 5, 15 min).

```
$ uptime
 10:21:02 up 3 days,  4:11,  1 user,  load average: 0,15 0,10 0,08
```

Carga 1.0 ≈ un núcleo al 100%. En un CPU de 4 núcleos, 4.0 es saturación.

lscpu / nproc

Información de la CPU y número de núcleos disponibles.

```
$ nproc
8
$ lscpu | head
```

nproc es el número que usan make -j y las herramientas de compilación.

df

Espacio de los sistemas de archivos montados. -h legible, -T tipo.

```
$ df -h
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda2    98G   42G   51G  46% /
```

Si Use% llega a 100% en / el sistema se vuelve inestable. df -i muestra inodos.

du

Cuánto ocupa una carpeta. -sh resume; -h lista; --max-depth=1 un nivel.

```
$ du -sh ~/practica
1,2M    /home/usuario/practica
$ du -h --max-depth=1 /var | sort -h
```

El truco para encontrar qué llena el disco: du -h --max-depth=1 / | sort -h.

vmstat

Resumen de procesos, memoria, swap, IO y CPU en intervalos.

```
$ vmstat 1 5
```

Si si (swap in) no para de subir, te falta RAM y el sistema está paginando.

sync

Fuerza a escribir a disco todo lo que está en caché.

```
$ sync
```

Úsalo antes de extraer un USB a lo bruto o antes de un apagado de emergencia.

dd

Copia datos a bajo nivel, bloque a bloque. Potente y peligroso.

```
$ dd if=/dev/zero of=prueba.bin bs=1M count=10
10+0 records in
10+0 records out
```

if = origen, of = destino. Un of=/dev/sda mal puesto borra un disco. Revisa dos veces.

CAPÍTULO 16

Discos, particiones y montaje

Un disco se ve como /dev/sdX o /dev/nvme0n1. Hay que particionarlo, formatearlo y montarlo para usarlo.

lsblk

Árbol de discos, particiones y puntos de montaje. El mapa más claro.

```
$ lsblk
NAME   SIZE TYPE MOUNTPOINTS
sda    100G disk
├─sda1   1G part /boot
└─sda2  99G part /
```

Empieza por lsblk antes de tocar fdisk. -f añade UUID y tipo de sistema de archivos.

blkid

UUID y tipo de cada partición. El UUID es lo que se pone en /etc/fstab.

```
$ sudo blkid
/dev/sda2: UUID="a1b2c3..." TYPE="ext4"
```

Montar por UUID no se rompe si cambia el orden de los discos.

mount / umount / findmnt

Monta o desmonta un sistema de archivos. findmnt lista lo montado con detalle.

```
$ sudo mkdir -p /mnt/usb
$ sudo mount /dev/sdb1 /mnt/usb
$ findmnt /mnt/usb
$ sudo umount /mnt/usb
```

Si umount dice busy, cierra procesos con fuser -km /mnt/usb (con cuidado).

fdisk / parted

Editores de tabla de particiones. fdisk es clásico; parted aguanta discos enormes y GPT.

```
$ sudo fdisk -l
Disk /dev/sda: 100 GiB
$ sudo parted -l
```

Solo lista con -l mientras estudias. Crear o borrar particiones destruye datos.

mkfs

Crea un sistema de archivos (formatea). Elige el tipo: ext4, xfs, vfat...

```
$ sudo mkfs.ext4 /dev/sdb1
```

Formatear borra la partición. Confirma el nombre con lsblk tres veces.

fsck

Comprueba y repara un sistema de archivos. Debe estar desmontado.

```
$ sudo umount /dev/sdb1
$ sudo fsck /dev/sdb1
```

Ubuntu lo lanza solo si un disco no se desmontó bien. No lo ejecutes sobre / en caliente.

CAPÍTULO 17

Red

Primero comprueba tu IP, luego si hay eco (ping), luego DNS y por último puertos.

ip

Herramienta moderna de red (sustituye ifconfig). ip a lista interfaces.

```
$ ip a
2: wlan0: <BROADCAST,UP> mtu 1500
    inet 192.168.1.40/24
```

ip a direcciones, ip r rutas, ip link set wlan0 up/down. ifconfig es legado.

hostname / hostnamectl

Nombre de la máquina. hostnamectl también muestra el sistema operativo.

```
$ hostname
ubuntu-pc
$ hostnamectl
```

sudo hostnamectl set-hostname nuevo-nombre cambia el nombre de forma persistente.

ping

Envía paquetes ICMP para ver si un host responde y con qué latencia.

```
$ ping -c 4 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=18.2 ms
```

-c 4 manda solo 4. Si 8.8.8.8 responde pero ubuntu.com no, el problema es DNS.

traceroute / tracepath

Muestra los saltos hasta el destino. tracepath no necesita root.

```
$ tracepath ubuntu.com
```

Sirve para ver en qué router se pierde la conexión.

ss / netstat

Puertos y conexiones. ss es el moderno; netstat puede no estar instalado.

```
$ ss -tuln
Netid  State   Local Address:Port
tcp    LISTEN  0.0.0.0:22
```

-t tcp, -u udp, -l listen, -n numérico (sin resolver nombres). -p añade el proceso (sudo).

dig / host / nslookup

Consultas DNS. dig es el más claro; host es rápido; nslookup es el clásico.

```
$ host ubuntu.com
ubuntu.com has address 185.125.190.20
$ dig +short ubuntu.com
```

Si ping a una IP funciona y host falla, tu DNS está mal (mira /etc/resolv.conf).

curl

Habla HTTP y muchos otros protocolos. Ideal para probar APIs y cabeceras.

```
$ curl -I https://ubuntu.com
HTTP/2 200
server: nginx
```

-I solo cabeceras, -O descarga, -L sigue redirecciones, -s silencioso.

wget

Descarga archivos. Más 'a lo descargador' que curl.

```
$ wget https://example.com/archivo.zip
```

wget -c reanuda una descarga cortada. -P /tmp elige destino.

nc (netcat)

Navaja suiza TCP/UDP: probar un puerto, escuchar o mandar un archivo.

```
$ nc -zv 127.0.0.1 22
Connection to 127.0.0.1 22 port [tcp/ssh] succeeded!
```

-z escanea sin mandar datos, -v verbose. No lo uses para escanear redes ajenas.

nmcli

NetworkManager por terminal. Wi-Fi, Ethernet y perfiles en Ubuntu de escritorio.

```
$ nmcli device status
wlan0  wifi  connected  MiWifi
$ nmcli connection show
```

nmcli device wifi list ve redes. nmcli radio wifi off apaga el Wi-Fi.

CAPÍTULO 18

Acceso remoto y transferencia

ssh entra a otra máquina. scp y rsync copian archivos. Nunca compartas tu clave privada.

ssh

Abre una sesión cifrada en otro equipo. Es el acceso estándar a servidores Linux.

```
$ ssh usuario@192.168.1.50
$ ssh -p 2222 usuario@servidor.ejemplo.com
```

Primera vez te pide confirmar la huella. ssh-copy-id instala tu clave para no usar contraseña.

scp

Copia archivos por SSH. Sintaxis origen destino, uno de los dos es remoto.

```
$ scp notas.txt usuario@servidor:/tmp/
$ scp usuario@servidor:/var/log/syslog ./
```

scp -r copia carpetas. Cada vez abre conexión nueva; para sync grande usa rsync.

sftp

FTP sobre SSH, interactivo: ls, put, get, cd en el servidor remoto.

```
$ sftp usuario@servidor
sftp> put notas.txt
sftp> ls
sftp> bye
```

Más cómodo que scp si vas a mover muchos archivos a mano.

rsync por SSH

Sincroniza una carpeta contra un servidor. Solo manda diferencias.

```
$ rsync -avz -e ssh curso/ usuario@servidor:~/curso/
```

-z comprime en tránsito. --progress muestra avance. --dry-run simula.

ssh-keygen

Crea un par de claves SSH. La pública se copia al servidor; la privada no sale de tu PC.

```
$ ssh-keygen -t ed25519 -C 'mi-correo@ejemplo.com'
$ ls ~/.ssh
id_ed25519  id_ed25519.pub
```

id_ed25519 es privada (600). .pub es pública y se puede compartir.

CAPÍTULO 19

Paquetes en Ubuntu (apt y dpkg)

Ubuntu instala software con apt, que usa el formato .deb de Debian. Snap es un canal aparte.

apt update

Actualiza la lista de paquetes disponibles. No instala nada. Hazlo antes de instalar.

```
$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
```

Si omites update, apt puede intentar versiones viejas o no encontrar un paquete nuevo.

apt upgrade / full-upgrade

Instala versiones nuevas de lo que ya tienes. full-upgrade puede quitar paquetes si hace falta.

```
$ sudo apt upgrade
$ sudo apt full-upgrade
```

upgrade es el día a día. full-upgrade antes de un salto grande de versión.

apt install / remove / purge / autoremove

Instala o quita paquetes. purge borra también la configuración. autoremove limpia dependencias huérfanas.

```
$ sudo apt install htop
$ sudo apt remove htop
$ sudo apt purge htop
$ sudo apt autoremove
```

adduser vs apt: adduser crea cuentas, apt instala programas. No los mezcles.

apt search / show / list

Busca paquetes, ve detalles y lista lo instalado.

```
$ apt search htop
$ apt show htop
$ apt list --installed | grep htop
```

search y show no necesitan sudo.

apt-get / apt-cache

Herramientas clásicas. En scripts se suele preferir apt-get por su salida estable.

```
$ sudo apt-get update
$ apt-cache policy htop
```

En la terminal interactiva usa apt. En cron y recetas, apt-get.

dpkg

Gestor de bajo nivel de .deb. apt lo llama por debajo.

```
$ dpkg -l | head
$ sudo dpkg -i paquete.deb
$ dpkg -L htop
```

-i instala un .deb local. Si faltan dependencias: sudo apt -f install.

add-apt-repository

Añade un PPA o un repositorio extra. Paquete: software-properties-common.

```
$ sudo add-apt-repository ppa:ejemplo/ppa
$ sudo apt update
```

Solo añade PPA de fuentes que te fíes. Luego se instala con apt install.

snap

Paquetes confinados de Canonical, independientes de apt.

```
$ snap list
$ sudo snap install vlc
$ snap find vlc
```

Algunas apps de Ubuntu (Firefox en varias versiones) llegan por snap, no por apt.

CAPÍTULO 20

systemd: servicios y arranque

systemd es el PID 1 de Ubuntu. Arranca servicios, los reinicia y guarda sus logs.

systemctl status / start / stop / restart

Consulta y controla un servicio (unidad).

```
$ systemctl status ssh
$ sudo systemctl restart ssh
$ sudo systemctl stop ssh
$ sudo systemctl start ssh
```

status no necesita sudo. Los otros sí, porque cambian el sistema.

systemctl enable / disable / is-enabled

enable hace que el servicio arranque con el sistema. disable lo quita del arranque.

```
$ systemctl is-enabled ssh
enabled
$ sudo systemctl enable nginx
$ sudo systemctl disable nginx
```

enable no lo inicia ahora (usa start). disable no lo para ahora (usa stop).

systemctl list-units

Lista unidades cargadas. --type=service solo servicios. --failed los que rompieron.

```
$ systemctl list-units --type=service --state=running
$ systemctl --failed
```

Si algo no arranca, --failed es el primer sitio que mirar.

journalctl

Lee el diario de systemd. Es el log unificado del sistema.

```
$ journalctl -u ssh -n 20
$ journalctl -xe
$ journalctl -f
```

-u unidad, -n últimas N, -f sigue, -xe errores recientes, --since today. sudo a veces hace falta.

hostnamectl / timedatectl / localectl

Configuran nombre, hora/zona y teclado/locale de forma persistente.

```
$ timedatectl
               Local time: mar 2026-09-22 10:30:00
$ timedatectl list-timezones | grep Madrid
$ sudo timedatectl set-timezone Europe/Madrid
```

localectl set-keymap es para consola. En escritorio el teclado lo lleva GNOME.

service

Envuelta antigua que en Ubuntu acaba llamando a systemctl.

```
$ sudo service ssh status
```

Funciona, pero aprende systemctl: es la interfaz real.

CAPÍTULO 21

Registros (logs)

Cuando algo falla, los logs cuentan la historia. En Ubuntu moderno vive sobre todo en journalctl.

dmesg

Mensajes del kernel: hardware, USB, discos, errores de drivers.

```
$ dmesg | tail
$ sudo dmesg -T | grep -i error
```

-T traduce marcas de tiempo a fecha humana. Suele requerir sudo.

logger

Escribe una línea en el log del sistema desde un script.

```
$ logger 'backup terminado'
$ journalctl -n 5 | grep backup
```

Así tus propios scripts dejan rastro junto al resto del sistema.

Archivos clásicos en /var/log

Aún existen logs en texto: syslog, auth.log, dpkg.log, apt/history.log.

```
$ sudo tail -n 20 /var/log/auth.log
$ less /var/log/dpkg.log
```

auth.log registra sudo, ssh e inicios de sesión. Léelo si sospechas un acceso.

CAPÍTULO 22

Fecha, hora y espera

La hora del sistema importa para cron, certificados y logs. Aprende a leerla y a pausar un script.

date

Fecha y hora actuales. Acepta formato con + y también puede fijar la hora (root).

```
$ date
mar 22 sep 2026 10:35:01 CDT
$ date +%Y-%m-%d
2026-09-22
```

+%H:%M hora. date -d 'yesterday' interpreta fechas. No cambies la hora si usas NTP.

cal

Calendario en texto.

```
$ cal
   Septiembre 2026
Do Lu Ma Mi Ju Vi Sá
       1  2  3  4  5
```

cal 2026 muestra el año. ncal -w añade número de semana.

sleep

Pausa N segundos (también 0.5 o 2m, 1h).

```
$ sleep 2 && echo listo
listo
```

Pieza básica de cualquier script que espera a otro servicio.

hwclock

Lee o sincroniza el reloj de hardware (la pila de la placa).

```
$ sudo hwclock --verbose
```

timedatectl es más amable. hwclock es el nivel bajo.

CAPÍTULO 23

Shell, variables e historial

Bash (la shell de Ubuntu) no es solo un sitio para pegar comandos: tiene variables, alias y memoria.

`echo $VARIABLE / printenv / env`

Las variables de entorno configuran programas (PATH, HOME, LANG, USER).

```
$ echo $HOME
/home/usuario
$ echo $PATH
/usr/local/bin:/usr/bin:/bin
$ printenv USER
```

PATH es la lista de carpetas donde bash busca comandos. Si 'no encuentra' un programa, mira PATH.

`export / unset`

export hace visible una variable a los programas hijos. unset la borra.

```
$ export SALUDO=hola
$ bash -c 'echo $SALUDO'
hola
$ unset SALUDO
```

Sin export, la variable solo existe en esa shell. Para que dure, ve a ~/.bashrc.

`alias / unalias`

Atajo de un comando largo. Ubuntu ya trae alias de ls con color.

```
$ alias ll='ls -lah'
$ ll
$ unalias ll
```

Ponlos en ~/.bashrc para que sobrevivan. alias solo lista los actuales.

`source / .`

Ejecuta un archivo en la shell actual (no en una subshell). Recarga configuración.

```
$ echo "export CURSO=linux" >> ~/.bashrc
$ source ~/.bashrc
$ echo $CURSO
linux
```

Después de editar .bashrc, source ~/.bashrc. Abrir otra terminal también vale.

history

Lista los comandos que ya ejecutaste. Se guarda en ~/.bash_history.

```
$ history | tail -n 5
  210  ls
  211  cd ~/practica
$ history | grep apt
```

↑ y ↓ recorren el historial. Ctrl+R busca incremental. !210 repite el número 210.

clear / reset

clear limpia la pantalla. reset repara una terminal que se puso rara (caracteres locos).

```
$ clear
$ reset
```

Ctrl+L también limpia. reset cuando un comando binario ensució la consola.

locale

Idioma y formato de fechas, números y moneda.

```
$ locale
LANG=es_MX.UTF-8
```

Si ves acentos rotos, casi siempre es un locale mal configurado.

set / shopt

Opciones de bash. set -e aborta un script al primer error. set -x depura.

```
$ set -x
$ echo hola
+ echo hola
hola
$ set +x
```

Al inicio de un script serio: set -euo pipefail.

CAPÍTULO 24

Redirecciones, tuberías y operadores

Esto no son comandos: son el pegamento. Si los dominas, dejas de ser principiante.

| tubería (pipe)

La salida de la izquierda es la entrada de la derecha. Encadena filtros.

```
$ ls /etc | grep ssh
ssh_config
sshd_config
```

Piensa en una cadena de montaje: cada comando hace una sola cosa y pasa el resultado.

> y >>

> escribe a un archivo y borra lo que había. >> añade al final.

```
$ echo uno > salida.txt
$ echo dos >> salida.txt
$ cat salida.txt
uno
dos
```

echo hola > archivo.txt con un solo > vacía el archivo. Es el accidente más común.

< entrada desde archivo

El comando lee el archivo como si lo hubieras tecleado.

```
$ wc -l < veinte.txt
20
```

Es distinto de wc -l veinte.txt: aquí wc no recibe el nombre, solo el contenido.

2> y 2>&1

2 es la salida de error. 2> errores.txt la guarda. 2>&1 junta error y salida normal.

```
$ ls no-existe 2> errores.txt
$ cat errores.txt
ls: no se puede acceder a 'no-existe': No existe el archivo
```

comando > todo.txt 2>&1 guarda las dos salidas. /dev/null es un agujero negro.

&& y ||

&& ejecuta lo siguiente solo si lo anterior salió bien. || solo si falló.

```
$ mkdir -p demo && cd demo && pwd
/home/usuario/practica/demo
$ ls falta || echo 'no estaba'
```

No uses ; si el segundo paso depende del primero. mkdir x; cd x es más frágil que &&.

; y &

; separa comandos y los corre igual. & lanza el de la izquierda en segundo plano.

```
$ date; echo hola; pwd
$ sleep 30 &
```

& devuelve el prompt al momento. Mira jobs para verlo. No lo uses con comandos interactivos.

$( ) y ` `

Sustitución de comando: se ejecuta lo de dentro y se pega el resultado.

```
$ echo "Hoy es $(date +%Y-%m-%d)"
Hoy es 2026-09-22
```

$( ) es la forma moderna. Evita los acentos graves ` ` en scripts nuevos.

Comodines * ? [ ]

La shell expande patrones de nombres de archivo antes de llamar al comando.

```
$ ls *.txt
frutas.txt  notas.txt
$ ls dia?.txt
$ ls archivo[12].txt
```

** cualquiera, ? un carácter, [12] 1 o 2. Entre comillas no se expanden: ls '*.txt'.*

CAPÍTULO 25

Tareas programadas

cron ejecuta algo en un horario. at lo ejecuta una vez. systemd timers es la vía moderna.

crontab

Edita la tabla cron del usuario. Cada línea: minuto hora día mes día-semana comando.

```
$ crontab -e
# Cada día a las 07:30
30 7 * * * /usr/bin/date >> /home/usuario/practica/hora.txt
$ crontab -l
```

Usa rutas absolutas. cron no carga tu .bashrc ni el mismo PATH de la terminal.

at

Programa un comando para que corra una vez, más tarde.

```
$ echo 'echo hola > ~/practica/at.txt' | at now + 2 minutes
$ atq
```

atq lista, atrm borra. El servicio atd debe estar activo.

systemd-run / timers

Alternative moderna a cron, integrada con journalctl.

```
$ systemd-run --on-active=10s echo hola
$ systemctl list-timers
```

Cuando un servicio ya es de systemd, un timer encaja mejor que crontab.

CAPÍTULO 26

Apagar, reiniciar y salir

Cierra siempre de forma limpia: los sistemas de archivos se pueden corromper si cortas la luz.

shutdown

Apaga o reinicia avisando a los usuarios. -h apaga, -r reinicia, now es inmediato.

```
$ sudo shutdown -h now
$ sudo shutdown -r +10 'Reinicio en 10 minutos'
$ sudo shutdown -c
```

-c cancela un apagado programado. Es la forma educada en un servidor con más gente.

reboot / poweroff / halt

Atajos. En Ubuntu acaban hablando con systemd.

```
$ sudo reboot
$ sudo poweroff
```

reboot = shutdown -r now. poweroff = shutdown -h now.

logout / exit

Cierran la sesión de terminal. exit también termina un script.

```
$ exit
```

Ctrl+D es el atajo de exit cuando la línea está vacía.

CAPÍTULO 27

Información del sistema y hardware

Cuando te sientas en una máquina nueva, estos comandos te dicen qué es y qué tiene dentro.

uname

Núcleo y arquitectura. -a es el resumen completo.

```
$ uname -a
Linux ubuntu-pc 6.8.0-40-generic #40-Ubuntu SMP x86_64 GNU/Linux
```

-s kernel, -r versión, -m arquitectura (x86_64, aarch64).

lsb_release / os-release

Qué distribución y versión de Ubuntu estás usando.

```
$ lsb_release -a
Description:  Ubuntu 24.04.1 LTS
$ cat /etc/os-release
```

Imprescindible antes de copiar instrucciones de internet: 22.04 y 24.04 no son iguales.

lshw / dmidecode

Inventario de hardware. dmidecode lee la BIOS/UEFI (DMI).

```
$ sudo lshw -short
$ sudo dmidecode -t memory
```

Suelen necesitar sudo. lshw -class disk lista discos con modelo.

lspci / lsusb

Dispositivos PCI (gráfica, red, NVMe) y USB conectados.

```
$ lspci
00:02.0 VGA compatible controller: ...
$ lsusb
```

Si un USB no aparece, prueba otro puerto y mira dmesg.

lsmod / modprobe

Módulos del kernel (drivers cargados). modprobe carga o descarga uno.

```
$ lsmod | head
$ sudo modprobe nombre_modulo
$ sudo modprobe -r nombre_modulo
```

No descargues módulos al azar: puedes perder red, disco o gráficos.

---

uname + who + df (foto rápida)

Rutina de 20 segundos al sentarte en un servidor desconocido.

```
$ uname -a && whoami && df -h && ip -br a
```

Kernel, tu usuario, disco y red. Con eso ya no estás ciego.

CAPÍTULO 28

Ayuda e información de comandos

Nadie se sabe todas las opciones. Lo profesional es saber dónde mirar.

man

Manual oficial. Secciones: 1 usuario, 5 archivos de config, 8 administración.

```
$ man chmod
$ man 5 passwd
$ man man
```

/palabra busca, n siguiente, q sale. man -k disco busca en las descripciones.

info

Manuales GNU, a veces más largos que man, con hiperenlaces.

```
$ info coreutils
```

Flechas para moverte, q sale. Menos usado, pero existe.

help

Ayuda de los builtins de bash (cd, alias, history, [, read...).

```
$ help cd
$ help history
```

man cd puede mostrarte el de otra shell. help es la verdad de bash.

whatis / apropos

whatis una línea de resumen. apropos busca esa palabra en todos los man.

```
$ whatis ls
ls (1)  - list directory contents
$ apropos 'disk space'
```

apropos es man -k. Ideal cuando no recuerdas el nombre del comando.

tldr

Ejemplos cortos y humanos. Se instala aparte y complementa a man.

```
$ sudo apt install tldr
$ tldr tar
```

Cuando man es un muro de texto, tldr te da los 5 usos reales.

CAPÍTULO 29

Otros coreutils que debes reconocer

Forman parte de GNU coreutils. Aparecen en exámenes, scripts ajenos y foros. No hace falta memorizarlos todos el primer mes.

`true / false`

No hacen nada útil por sí solos: true sale con código 0, false con 1.

```
$ true && echo ok
ok
$ false || echo fallo
fallo
```

Se usan en scripts y en condiciones. echo $? muestra el código del último comando.

`yes (recordatorio)`

Ya lo viste: genera 'y' infinito. Aquí el uso típico de confirmación.

```
$ yes | apt list --installed | head
```

En instalaciones desatendidas a veces verás yes | comando.

`factor`

Descompone un número en factores primos.

```
$ factor 84
84: 2 2 3 7
```

Ejemplo de utilidad GNU pequeña. Buen juguete, poco uso de administración.

`numfmt`

Convierte números a unidades humanas o al revés.

```
$ echo 1048576 | numfmt --to=iec
1.0M
```

Muy útil al formatear salidas de du o de scripts de cuotas.

basename / dirname (en scripts)

Patrón típico para localizar la carpeta del propio script.

```
$ archivo=/home/usuario/practica/a.txt
$ echo "$(dirname $archivo)/copia-$(basename $archivo)"
/home/usuario/practica/copia-a.txt
```

En scripts robustos se citan las variables: "$(dirname "$archivo")".

install -d

Crea carpetas como mkdir -p, con permisos explícitos.

```
$ install -d -m 750 ~/practica/privado
$ ls -ld ~/practica/privado
drwxr-x--- 2 usuario usuario 4096 ...
```

Lo verás en Makefiles y recetas de empaquetado.

pr / fmt / fold

Dan formato a texto: páginas, relleno de párrafos y ajuste de ancho.

```
$ echo 'un texto bastante largo para la terminal' | fold -w 20
un texto bastante l
argo para la termin
al
```

fmt rellena párrafos. pr pagina para imprimir. Hoy se usan poco, pero existen.

expand / unexpand

Pasan tabuladores a espacios y al revés.

```
$ printf 'a\tb\n' | expand -t 4
a   b
```

Útil al limpiar código o archivos TSV mal formateados.

join

Une líneas de dos archivos por un campo común. Ambos deben estar ordenados.

```
$ echo -e '1 ana\n2 luis' > p.txt
$ echo -e '1 9\n2 8' > n.txt
$ join p.txt n.txt
1 ana 9
2 luis 8
```

Si no están ordenados, join se equivoca. Ordénalos antes con sort.

tsort

Orden topológico: resuelve dependencias (A antes que B).

```
$ printf 'a b\nb c\n' | tsort
a
b
c
```

Aparece en recetas de compilación y grafos de paquetes.

uniq (con sort)

Patrón de estudio: contar qué se repite más.

```
$ printf 'error\ninfo\nerror\nerror\ninfo\n' | sort | uniq -c | sort -nr
      3 error
      2 info
```

sort | uniq -c | sort -nr es de las tuberías más útiles que existen.

stdbuf

Cambia el buffer de un comando. Evita que la tubería 'se quede muda' hasta llenarse.

```
$ stdbuf -oL grep error grande.log | head
```

-oL salida línea a línea. Lo necesitas cuando tail -f | grep no imprime al momento.

timeout + yes (patrón de examen)

Combina dos coreutils para no dejar un comando infinito suelto.

```
$ timeout 1 yes | wc -l
185492
```

yes solo no para. timeout lo corta. El número de líneas depende de lo rápido que sea el PC.

CAPÍTULO 30

Plan de 7 días

No intentes memorizar la guía de un tirón. Un capítulo bien practicado vale más que diez leídos.

Día 1 — Navegar

pwd, ls, cd, tree. Entra y sal de /etc, /var/log y tu home hasta hacerlo sin mirar.

```
$ mkdir -p ~/practica && cd ~/practica
$ pwd
$ ls -la
$ cd /etc && ls && cd -
```

Meta: llegar a cualquier ruta y volver a ~/practica en menos de 10 segundos.

Día 2 — Archivos

touch, mkdir, cp, mv, rm, cat. Crea, copia, renombra y borra solo dentro de ~/practica.

```
$ touch notas.txt
$ mkdir backup
$ cp notas.txt backup/
$ mv backup/notas.txt backup/copia.txt
```

Meta: explicar la diferencia entre cp y mv, y entre > y >>.

Día 3 — Leer y buscar

less, head, tail, grep, find. Busca texto en tus archivos y en /var/log.

```
$ echo -e 'manzana\npera' > frutas.txt
$ grep pera frutas.txt
$ find ~/practica -name '*.txt'
```

Meta: filtrar la salida de ls o ps con grep sin dudar.

Día 4 — Permisos y usuarios

ls -l, chmod, chown, whoami, id, sudo. Lee rwx y cambia 644 / 755.

```
$ touch secreto.txt
$ chmod 600 secreto.txt
$ ls -l secreto.txt
$ whoami && id
```

Meta: decir en voz alta qué significa rwxr-xr-x y por qué 777 es mala idea.

Día 5 — Procesos y recursos

ps, top, kill, free, df, du. Abre sleep 300 y mátalo por PID.

```
$ sleep 300 &
$ ps aux | grep sleep
$ free -h && df -h
```

Meta: identificar un proceso, su PID y terminarlo con kill (sin -9).

Día 6 — Paquetes Ubuntu

apt update, search, install, show, remove. Instala htop y tree, úsalos y decide si los dejas.

```
$ sudo apt update
$ apt search htop
$ sudo apt install htop tree
$ htop
```

Meta: instalar un programa y saber cómo desinstalarlo después.

Día 7 — Red, tuberías y examen

ip, ping, curl, |, >, >>. Encadena tres comandos y guarda el resultado.

```
$ ip -br a
$ ping -c 2 8.8.8.8
$ ls -la | grep txt > resultado.txt
$ cat resultado.txt
```

Examen: tres .txt, grep de una palabra, resultado guardado. Si lo haces sin mirar, ya no eres cero.

CAPÍTULO 31

Errores comunes y reglas de oro

Casi todos los sustos de principiantes caben en esta lista. Léela dos veces.

rm -rf fuera de práctica

Borra sin preguntar y no hay papelera. Un espacio mal puesto cambia el destino.

```
$ # PELIGRO, no lo ejecutes:
$ # rm -rf / home/usuario/tmp
$ #           ^ ese espacio borra /
```

*Regla: escribe primero la ruta, revisa con ls, y solo entonces pon rm. Nunca rm -rf * en /.*

Un solo > en vez de >>

Pisa el archivo y deja solo la última escritura.

```
$ echo importante > bitacora.txt
$ echo otra cosa > bitacora.txt
$ cat bitacora.txt
otra cosa
```

Si quieres histórico, usa >> o tee -a. Si vas a usar >, mira el nombre dos veces.

chmod 777

Todo el mundo puede leer, escribir y ejecutar. En un servidor es una invitación.

```
$ # Evita esto:
$ # chmod 777 -R /var/www
$ chmod 755 script.sh
$ chmod 644 archivo.txt
```

Scripts: 755. Archivos de datos: 644. Secretos: 600. Carpetas privadas: 700.

sudo por costumbre

sudo no es un prefijo mágico. Te deja romper el sistema más rápido.

```
$ # Primero sin sudo. Si el error es permiso, entonces sudo.
$ systemctl status ssh
$ sudo systemctl restart ssh
```

Si un comando de usuario funciona, no lo conviertas en root 'por si acaso'.

cd a una ruta y olvidar dónde estás

Borras o editas en el sitio equivocado porque no miraste pwd.

```
$ pwd
$ ls
$ cd ~/practica
```

Antes de rm, chmod o > : pwd && ls. El hábito vale más que cualquier alias.

Espacios en nombres de archivo

La shell parte por espacios. Sin comillas, un nombre se convierte en varios argumentos.

```
$ mkdir 'mi carpeta'
$ ls mi carpeta
ls: no se puede acceder a 'mi': No existe
$ ls 'mi carpeta'
```

Usa comillas o escapa: mi\ carpeta. En scripts, siempre "$variable".

PATH y 'comando no encontrado'

El programa puede estar instalado y aun así no verse si su carpeta no está en PATH.

```
$ type codigo
$ echo $PATH
$ ls /usr/bin/codigo /usr/local/bin/codigo
```

which y type diagnostican. No copies export PATH de internet sin entender qué quitas.
